Raio-X Comercial · Documentação do projeto

Projeto final da trilha IA Aplicada (SENAI/TCS) · Equipe: Marcos (dados), Eduardo (IA e gestão), Náthally (produto e métricas), Miguel (tela e infraestrutura) · Repositório: `MiguelGrispan/trabalho-tcs` (GitHub)

Versão: rascunho 1, qua 04/11/2026 · Final prevista: seg 09/11/2026 · Entrega: seg 16/11/2026

> **Rascunho.** As partes marcadas com **[PENDENTE]** esperam um entregável de outra pessoa (BPMN, diagrama de arquitetura, texto final de ética, ROI). A lista completa está no fim (Apêndice B) e sai da versão final.

O que este documento cobre

seção	o que o professor pede	fonte
1. O problema e o produto	contexto do tema e do objetivo	`requisitos_projeto.MD`, Documentação
2. Etapas desenvolvidas	"um documento ... explicando as etapas desenvolvidas durante o processo"	`requisitos_projeto.MD`, Documentação
3. Dados: limpeza e indicadores	Python e Pandas para limpeza, nulos e estruturação; SQL	`guia_projeto_final.pdf` §2
4. Processo (BPMN 2.0)	"Fluxograma BPMN 2.0 do processo"	guia §6; `requisitos_projeto.MD`, Documentação
5. Arquitetura e infraestrutura	arquitetura; "Infraestrutura automatizada com Terraform"	guia §2 e §6; `requisitos_projeto.MD`, Aplicação
6. Justificativa técnica	"Justificativa técnica das escolhas de modelos/arquiteturas (ex.: RAG vs Fine-tuning)"	guia §6 e §7
7. Ética, governança e segurança	"análise de ética e segurança"; viabilidade, limitações, viés, confiabilidade e uso responsável	guia §2, §5 e §6
8. Métricas	métricas de sucesso	guia §3 (bloco 5)
9. Como usamos IA para programar	agentic coding; arquivos de prompts do vibecoding	guia §2; `requisitos_projeto.MD`, Aplicação

Os dois arquivos do professor estão no repositório, sem mudança, em `docs/professor/`.

---

1. O problema e o produto

Quem usa: o executivo de contas B2B, que vende produtos ou serviços para empresas grandes.

A dor: antes da primeira reunião com uma companhia, ele precisa entender o tamanho dela, se cresce, se a margem está apertada e se está endividada. Hoje ele abre o site de relações com investidores (RI), procura os números à mão, tenta comparar com o setor e pensa em perguntas. Leva tempo, cada vendedor faz de um jeito e a comparação com empresas parecidas quase nunca acontece.

O que o Raio-X entrega. O vendedor escolhe uma companhia aberta da CVM, diz o que vende e o objetivo da reunião, e recebe em segundos:

- 6 indicadores das demonstrações financeiras entregues à CVM (DFP), cada um com a conta de onde veio;
- a posição de cada indicador contra o grupo de empresas de perfil parecido (K-means);

- 3 a 5 perguntas para a reunião, ligadas ao produto, e o que os números não mostram (lacunas);
- 5 empresas parecidas para prospectar (vizinhos mais próximos).

A regra de ouro. O texto é escrito por um LLM (Gemini, via LangChain), mas **o LLM nunca escreve números**. Ele usa marcadores como `{margem_ebit}`; um validador remove qualquer trecho com número fora de marcador; o código troca os marcadores pelos valores calculados da tabela. Número inventado num material de vendas é o pior erro possível, e esta regra o elimina por construção.

[PENDENTE: ROI · NA3, Náthally] *Tempo para montar um raio-x à mão (1 empresa, pelo site de RI) contra o tempo com a ferramenta, contando digitar, ler e conferir, e a conta do ganho em 3 linhas.*

---

2. Etapas desenvolvidas

O projeto durou 4 semanas (19/10 a 16/11/2026). O detalhe de cada passo está nos pull requests do repositório (`gh pr list --state all`) e o resumo diário em `docs/HISTORICO.md`.

Semana 1 (19 a 25/10): cada um no seu canto

- **Repositório criado** (19/10) e primeiras entregas separadas: produto e regras do texto (Náthally), carga e limpeza da DFP (Marcos, numa branch própria) e a primeira versão do Raio-X com IA, segmentação, checador de dados, tela e testes (Eduardo, PR #1).
- A **regra de ouro** nasce no PR #1 (21/10), junto com o `REGRAS-IA.md`, as regras do time para o uso de IA.

Semana 2 (26/10 a 01/11): juntar tudo e fazer funcionar

- **Fundação** (PR #2, 26/10): uma pasta por dono e o código do Marcos trazido para a estrutura comum.
- **Integração contínua** (27/10): testes automáticos em todo PR, versões fixas (Python 3.14) e os primeiros 3 raio-x com dados reais (595 empresas). O teste revelou um problema: o K-means pôs 567 das 595 empresas num grupo só.
- **Segmentação consertada** (PR #9, 28/10): receita mínima de R$ 10 mi, extremos cortados e menor grupo com 5% ou mais. A silhueta caiu de 0,77 para 0,30, mas os grupos passaram a ser úteis (seção 6).
- **Prompt da Náthally no código** (PR #11, 29/10): o Gemini lê o arquivo de regras dela, sem cópia. **Avaliação com 10 casos reais** (PR #20): 0 trechos removidos pelo validador (seção 8).
- **PostgreSQL** (PRs #21 e #22, 29 e 30/10): o professor fixou o PostgreSQL em aula (SQLite não vale); o CI passou a subir um banco só para os testes, para nenhum teste de banco ser pulado.
- **Docker** (PR #24, 30/10): `docker compose up` sobe a tela e o PostgreSQL juntos.
- **Terraform** (PR #29, 31/10): `terraform apply` sobe a mesma infraestrutura; o `destroy` mantém o banco e os dados.
- **Travas para a IA de programação** (PR #30, 31/10): regras que custam caro quebrar viraram bloqueios automáticos (seção 9).

Semana 3 (02 a 08/11): fechar o código e escrever

- **Justificativa técnica** (PR #31, 02/11) e **tela ligada ao banco** (PR #36, 02/11): a tela busca as empresas no PostgreSQL, grava cada consulta e, se o Gemini falhar, cai sozinha no modo sem IA.
- **Demo ponta a ponta** (03/11): Vale, Embraer e Ambev sem IA e WEG com o Gemini (US$ 0,0011), pelo `docker compose` e pelo Terraform, com tudo gravado no banco. Tag `v0.1-demo`.
- **Correção do ano fiscal** (PR #46, 03/11): a Camil fecha o exercício em fevereiro, e o sistema mostrava o exercício anterior. O "ano" de cada número passou a ser o do exercício. 5 empresas conferidas contra o RI, com diferença de 0,00% (seção 8).
- **Decisões de 04/11:** a demo da banca roda no Mac do Eduardo, o único conferido com o compose, o Terraform e o Gemini, e os vídeos são gravados nele; este documento passa a ser escrito em markdown e convertido em PDF.

- Código congelado e tag `v1.0` na sex 06/11; vídeo final gravado na `v1.0` no sáb 07/11. **[PENDENTE: confirmar as datas reais no fim da semana]**

Semana 4 (09 a 16/11): entregar e ensaiar

- Documento, deck e vídeo da demo prontos em qua 11/11; ensaios cronometrados; entrega em seg 16/11; banca em qua 18/11.

As decisões que mais mudaram o projeto

data	decisão	por quê
21/10	o LLM nunca escreve números; usa marcadores	número inventado num material de vendas é o pior erro possível
26/10	cada pasta tem um dono	quatro pessoas no mesmo arquivo geravam conflito
28/10	K-means só com receita a partir de R$ 10 mi, extremos cortados, menor grupo com 5% ou mais	sem isso, um grupo tinha 567 das 595 empresas
28/10	"empresas parecidas" = mesmo perfil financeiro, não o mesmo setor	é o que o modelo mede
29/10	banco PostgreSQL	regra do professor: SQLite não vale
30/10	Terraform obrigatório	`requisitos_projeto.MD`, Aplicação: "Infraestrutura automatizada com Terraform"
30/10	o Gemini desiste em até uns 30 s (15 s por tentativa, 2 tentativas)	sem limite, a API travada deixava a tela girando, sem chegar ao plano B
31/10	a chave do Gemini não passa pelo Terraform	variável do Terraform fica em texto puro no `terraform.tfstate`
02/11	sem RAG vetorial, sem fine-tuning, sem memória e sem agente	seção 6
02/11	a aplicação é o Streamlit, em 2 contêineres (tela + banco)	resposta do professor: "um contêiner para ele e a aplicação e um para o SQL"
03/11	o "ano" de cada número é o do exercício, e não o do arquivo da CVM	a Camil aparecia com o exercício anterior

---

3. Dados: limpeza e indicadores

A fonte

Dados abertos da CVM, em CSV: o cadastro das companhias abertas (`cad_cia_aberta.csv`: código, nome, situação e setor) e os arquivos anuais da DFP (`dfp_cia_aberta_AAAA.zip`). Da DFP usamos a DRE (resultado), o BPA (ativo) e o BPP (passivo e patrimônio), consolidados e individuais. São baixados os exercícios de 2023 a 2025, mais o arquivo de 2026, onde está o exercício 2025 de quem fecha o ano fora de dezembro (`scripts/carregar_dfp.py`).

Regras de limpeza, na ordem

As regras 1 a 7 estão no código com o mesmo número, em `scripts/carregar_dfp.py` (regra 1) e `raiox/limpeza.py` (as outras).

1. **Ler** com `sep=";"`, `encoding="latin-1"`, `decimal=","` e `CD_CONTA` como texto. Lido como número, o código de conta `3.10` viraria `3.1`. (`ler_demo()`, `scripts/carregar_dfp.py`)
2. **Só o exercício do ano** (`ORDEM_EXERC == "ÚLTIMO"`): a CVM manda também o ano anterior, e cada conta apareceria duas vezes.

3. **Só a maior versão** de cada empresa: reapresentações substituem a entrega original. Se a mesma versão traz dois períodos, fica o fim de exercício mais recente. (`limpar_demo()`)
4. **Escala:** `ESCALA_MOEDA = MIL` vira × 1.000. Além disso, o código acha empresas que declaram "unidade" mas mandam milhares: a receita cai cerca de 1.000 vezes de um ano para o outro e, multiplicada por 1.000, fica parecida com a do ano anterior. Esses anos são corrigidos e o script avisa. (`encontrar_escala_errada()` e `corrigir_escala()`)
5. **Consolidado primeiro:** sem demonstração consolidada, usa a individual e anota a origem (`con` ou `ind`). (`preferir_consolidado()`)
6. **Só empresas com receita de venda positiva** no último ano: bancos e seguradoras usam a conta 3.01 para outras receitas (ex.: "Receitas da Intermediação Financeira") e ficam de fora. (`filtrar_empresas_com_receita()`)
7. **Nulos: não inventar.** Valor que falta fica vazio, e a tela mostra "sem dado". Se a mesma conta aparece duas vezes com valores diferentes, o valor fica vazio em vez de escolher um (`resolver_conflitos()`). Divisão por zero ou por valor negativo deixa o indicador vazio em vez de infinito (`montar_indicadores()`, `scripts/indicadores.py`).

O ano do exercício (PR #46). A CVM arquiva cada DFP pelo ano em que o exercício termina, mas nem toda empresa fecha em dezembro. A Camil fecha em fevereiro: o "2025" dela vai de mar/2025 a fev/2026. O ano de cada número passou a ser aquele em que cai a maior parte dos meses do exercício (`ano_do_exercicio()`, `raiox/limpeza.py`). Mudaram as 9 empresas que fecham fora de dezembro; a receita da Camil foi de R$ 12,26 bi (o exercício anterior) para R$ 11,12 bi, o valor do RI.

Os 6 indicadores

Uma linha por empresa, no último exercício (`montar_indicadores()`, `scripts/indicadores.py`; formato no contrato C2, `contratos.md`).

indicador	fórmula	contas da DFP	o que diz ao vendedor
Receita líquida	valor da conta, último ano	DRE 3.01	tamanho da empresa
Crescimento em dois anos	receita do último ano ÷ receita de 2 anos antes – 1	DRE 3.01	está expandindo ou encolhendo
Margem EBIT	resultado antes do resultado financeiro e dos tributos ÷ receita	DRE 3.05 ÷ 3.01	pressão de custo na operação
Margem líquida	lucro do período ÷ receita	DRE 3.11 ÷ 3.01	o que sobra depois de juros e impostos
Ativo total	valor da conta, último ano	BPA 1	tamanho pelo que a empresa tem
Endividamento	1 – patrimônio líquido ÷ ativo total	BPP 2.03 e BPA 1	quanto depende de terceiros

Onde os dados ficam (SQL)

O PostgreSQL 17 guarda as tabelas `contas` (uma linha por empresa × ano × demonstração × conta, com índice único que recusa conta repetida), `empresas` (do cadastro), `indicadores` (os 6 indicadores) e `consultas` (cada raio-x pedido na tela: empresa, produto, objetivo, tempo e custo). A busca da tela usa `ILIKE` e devolve só companhias ativas e com indicadores (`raiox/dados.py`). Se o banco estiver desligado, a conexão desiste em 5 s e a tela mostra o que fazer, em vez de travar (`raiox/banco.py`).

Resultado da carga (Marcos, 03/11): 384.591 linhas de contas e 595 empresas com os 6 indicadores, em 1 min 48 s.

Auditoria dos dados

`raiox/checagem.py` aplica regras de sanidade (ex.: margem acima de 100%, receita que muda demais) e **só aponta**: quem decide é uma pessoa. Com os dados de 03/11: nenhum "erro" e 180 "atenção" em 98 empresas.

[PENDENTE: EDA · MA4, Marcos] *Os 4 gráficos da análise exploratória (nulos, distribuições, extremos, cobertura), de `notebooks/eda.ipynb`, com uma frase de leitura cada.*

---

4. Processo de negócio (BPMN 2.0)

[PENDENTE: BPMN · MA5, Marcos, até qui 05/11] *Imagem `docs/bpmn/` (`.png`), feita no bpmn.io.*

O diagrama mostra o processo da regra de negócio com o Raio-X, com as raias **Vendedor**, **Sistema** e **LLM (API externa)**:

1. o vendedor busca a empresa e informa o produto e o objetivo;
2. decisão **"a empresa tem dados?"**: se não, o sistema avisa e o processo acaba;
3. o sistema monta os indicadores, a posição no grupo e as empresas parecidas;
4. o LLM escreve o texto com marcadores;
5. decisão **"o texto passou no validador?"**: o trecho com número fora de marcador é removido e registrado;
6. o sistema troca os marcadores pelos valores e grava a consulta;
7. o vendedor revisa antes de usar (revisão humana no fim).

Se a API falhar, o sistema monta o texto sem IA, com os mesmos números (seção 5).

---

5. Arquitetura e infraestrutura

[PENDENTE: diagrama · MG5, Miguel] *Imagem `docs/arquitetura.png`.*

Os componentes

```
             PREPARAÇÃO (roda uma vez; refaz quando sai DFP nova)
CVM (cadastro + DFP 2023–2026)
   │  scripts/carregar_dfp.py     baixa, limpa (raiox/limpeza.py)        → PostgreSQL: contas, empresas
   ▼
scripts/indicadores.py            6 indicadores por empresa (C2)         → PostgreSQL: indicadores
   │
   ▼
raiox/segmentos.py                K-means + 5 vizinhos mais próximos     → grupo e parecidas de cada empresa

             CONSULTA (a cada uso, alguns segundos)
Vendedor ──► app/streamlit_app.py (tela)
                │  busca a empresa ............... raiox/dados.py (C1) ──► PostgreSQL
                │  pede o raio-x ................. raiox/ia.py (C3)
                │      números e posição: código
                │      texto: Gemini (API) via LangChain, só com marcadores
                │      validador: remove número fora de marcador
                │      sem IA ou API fora do ar: texto por modelo fixo
                └► mostra o raio-x e grava a consulta ──► PostgreSQL: consultas
```

Os formatos entre as partes (C1 a C4) estão no `contratos.md`: cada um trabalhou contra dados falsos no formato combinado (`falsos/`) até o colega entregar.

Dois contêineres, dois caminhos para subir

A aplicação roda em **2 contêineres**, como o professor orientou: um com a tela e a aplicação (Streamlit + `raiox/`) e outro com o PostgreSQL. O Streamlit faz o papel de frontend (a tela) e o código de `raiox/` o de backend.

	`docker compose`	**Terraform (obrigatório)**
arquivo	`docker-compose.yml`	`infra/*.tf`, com o provider Docker (`kreuzwerker/docker`)
sobe	`docker compose up -d --build`	`terraform apply`
derruba	`docker compose down`	`terraform destroy` (o banco e a `data/` continuam)
conferido	Linux (Miguel, 30/10) e Mac (Eduardo, 03/11)	Linux (Miguel, 31/10) e Mac (Eduardo, 03/11): 2º `plan` sem mudanças

O Terraform roda localmente, no Docker do computador (`requisitos_projeto.MD`, Aplicação: "podendo rodar localmente ou na nuvem"). O volume do banco fica fora do Terraform, para o `destroy` não apagar os dados carregados. O CI confere o Terraform (`terraform fmt` e `terraform validate`) em todo PR que mexe em `infra/`.

Tecnologias

parte	tecnologia
linguagem	Python 3.14, versões fixas no `requirements.txt`
dados	Pandas 3.0, NumPy 2.5; dados abertos da CVM
banco	PostgreSQL 17, SQLAlchemy 2.1, psycopg 3.3
machine learning	scikit-learn 1.9 (KMeans, NearestNeighbors, StandardScaler)
IA generativa	Gemini `gemini-3.1-flash-lite` por API, LangChain, saída estruturada com Pydantic
tela	Streamlit 1.66
infraestrutura	Docker, Docker Compose, Terraform 1.17
testes e CI	pytest 9.1, GitHub Actions com um PostgreSQL só para os testes

---

6. Justificativa técnica

6.1 O problema decide a arquitetura

O vendedor precisa de **números certos** e de **texto útil**. Cada parte do sistema faz o que faz melhor:

quem	faz	por quê
código (Pandas)	calcula todos os números a partir da DFP	cálculo é determinístico e auditável: cada número tem conta e ano
ML (scikit-learn)	acha o grupo de empresas parecidas e as 5 mais próximas	não há rótulo de "empresa parecida": é aprendizado não supervisionado
LLM (Gemini)	escreve o texto com marcadores, nunca o número	escrever é o que o LLM faz bem; copiar e arredondar número é onde ele erra
validador (código)	remove o trecho com número fora de marcador e troca os marcadores pelos valores	é a regra de ouro

6.2 Machine learning: K-means + vizinhos mais próximos

- **Não supervisionado** porque não existe uma coluna "grupo certo" para treinar: o K-means descobre os grupos sozinho.
- **5 variáveis:** log da receita, crescimento em dois anos, margem EBIT, margem líquida e endividamento. O log porque a receita vai de milhões a centenas de bilhões; sem ele, só o tamanho decidiria o grupo.
- **Preparo:** extremos cortados nos percentis 5 e 95, nulos preenchidos com a mediana (só para o modelo; a tela mostra o valor real ou "sem dado") e `StandardScaler`, para todas as variáveis ficarem na mesma régua.

- **Treino só com receita a partir de R$ 10 mi:** abaixo disso estão 31 empresas, 52% delas com margem EBIT acima de 100% ou abaixo de −100% (holdings e empresas sem operação); acima, 564 empresas e 4% (`scripts/avaliar_corte_receita.py`, 03/11). As empresas menores recebem o grupo mais próximo, sem participar do treino.
- **Escolha do k:** testa k de 3 a 8 e fica com a maior silhueta, **mas só entre os k em que o menor grupo tem pelo menos 5% das empresas** (`escolher_k()`, `raiox/segmentos.py`).
- **5 parecidas:** `NearestNeighbors` com 6 vizinhos no mesmo espaço padronizado, porque o 1º vizinho é a própria empresa.

Resultado (dados de 30/10): k = 4, silhueta 0,29, grupos de 327, 76, 59 e 102 empresas no treino (564 empresas).

Por que a silhueta de 0,77 enganava. Antes da regra dos 5%, a silhueta era 0,77 e parecia ótima, mas o modelo só separava 9 empresas extremas e juntava todas as outras num grupo só: 567 das 595. Com a regra, a silhueta cai para 0,30 (0,29 com os dados de 30/10) e os grupos passam a servir para comparar. Preferimos grupos úteis a um número bonito.

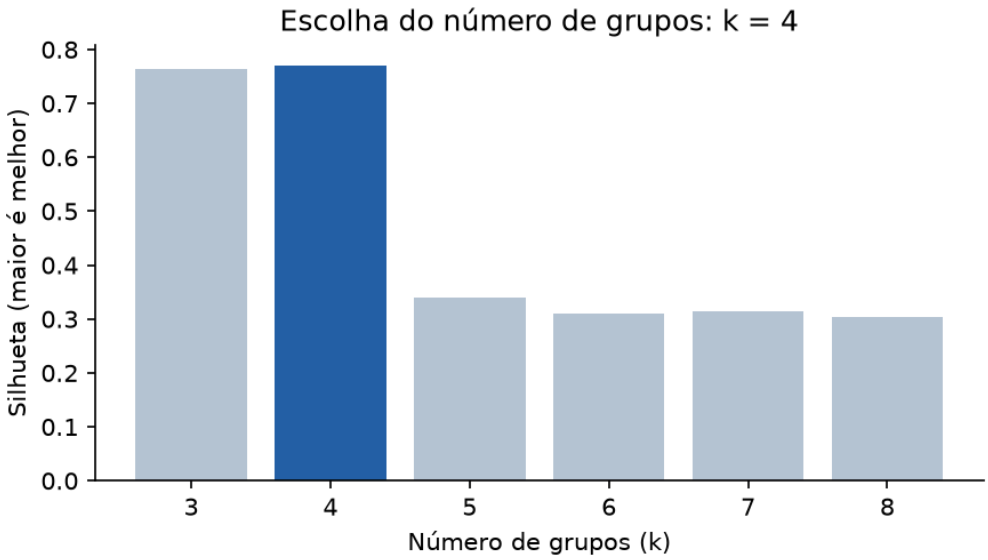


Figura 1. Silhueta por k antes da regra dos 5% (27/10): 0,77 em k = 4, com quase todas as empresas num grupo só.

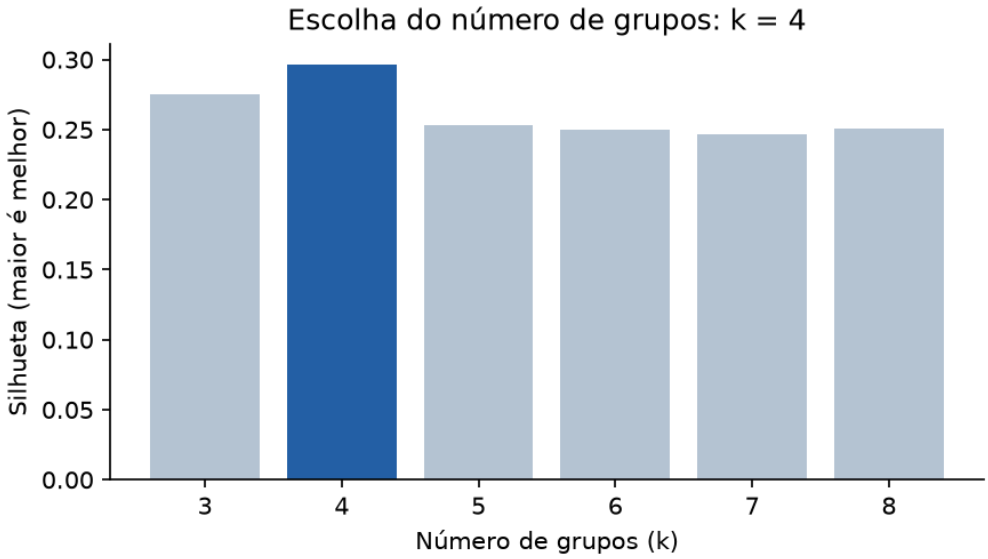


Figura 2. Silhueta por k depois da regra dos 5% (28/10): 0,30 em k = 4.

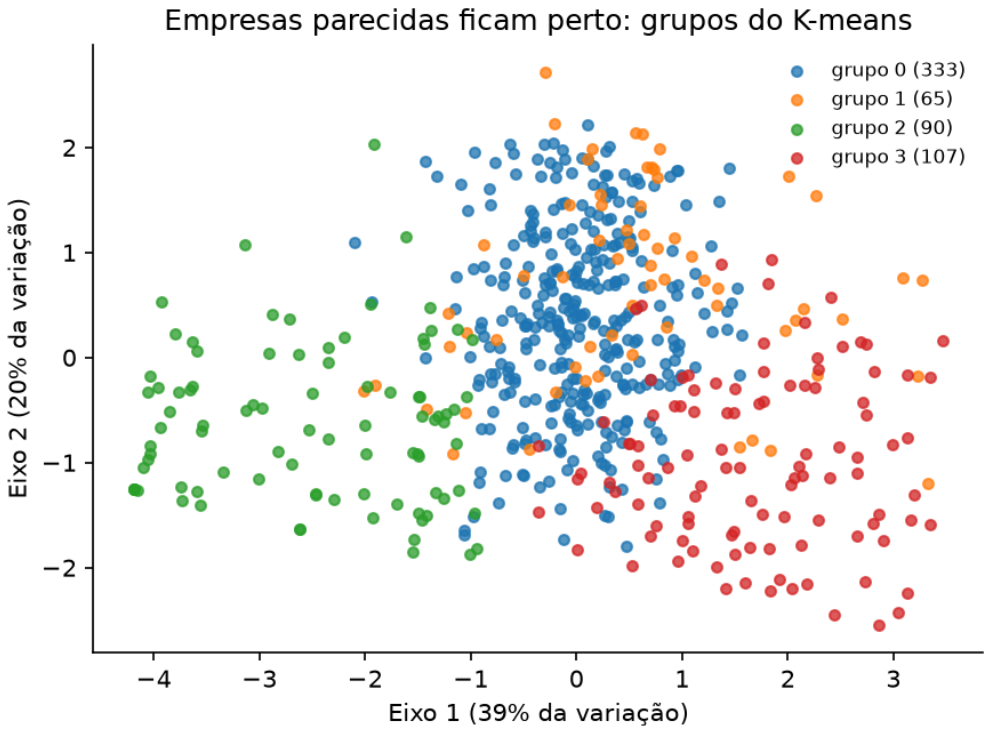


Figura 3. Os 4 grupos projetados em 2 dimensões (PCA), com os dados de 28/10. As 5 variáveis viram 2 eixos só para desenhar; o modelo usa as 5.

[PENDENTE: refazer as figuras 2 e 3 com os dados depois da correção do ano fiscal (PR #46). As de hoje são de 28/10: silhueta 0,30 e grupos de 333, 65, 90 e 107 na legenda (todas as empresas); o texto acima usa os dados de 30/10.]

6.3 IA generativa: por que nem RAG vetorial nem fine-tuning

A base de conhecimento da IA é a tabela de indicadores calculada da DFP da CVM. O código busca a linha exata da empresa pelo código CVM (`montar()`, `raiox/ia.py`) e entrega ao Gemini, para cada indicador, o nome, o valor da empresa, a mediana do grupo (já formatados, ex.: "R$ 12,3 bi") e a posição ("acima", "abaixo" ou "em linha"), calculada por código. O Gemini **lê** esses valores para entender a empresa, mas **só pode escrever os marcadores**. É geração aumentada por recuperação, com a recuperação **estruturada** (pela chave da empresa), e não semântica (por embeddings).

opção	o que seria	por que não usamos
RAG vetorial	quebrar documentos em pedaços, gerar embeddings e buscar os trechos mais parecidos com a pergunta	os dados são números em tabela, não texto. Busca por semelhança pode trazer a empresa ou o ano errado; busca por chave traz sempre a linha certa. Exigiria também um banco vetorial a mais
Fine-tuning	treinar o modelo com exemplos de raio-x bons	não temos centenas de exemplos rotulados; custa mais; os dados mudam todo ano; e treinar **não impede** o modelo de errar número, que é o risco principal
O que usamos	prompt + saída estruturada (Pydantic) + validador	barato (≈ US$ 0,0012 por raio-x), reproduzível e conferível

Quando o RAG passaria a valer: se o raio-x tivesse de ler texto livre, como o release de resultados ou as notas explicativas da DFP.

6.4 Chain, memória e agentes no LangChain

- **Uma chain de dois passos** (`_chain()`, `raiox/ia.py`): `ChatPromptTemplate` (as regras como mensagem de sistema + o pedido) → `ChatGoogleGenerativeAI(...).with_structured_output(RaioX)`.
- **Saída estruturada:** a resposta volta no formato `RaioX` (resumo, perguntas e lacunas), garantido pelo Pydantic.
- **O prompt de sistema** é o arquivo de regras da Náthally (`avaliacao/regras_prompt.md`), lido sem cópia, mais 3 notas do código. Assim, o que a dona das regras escreve chega inteiro ao modelo.
- **Modelo pequeno** (`gemini-3.1-flash-lite`, `temperature=0.2`): a tarefa é escrever 3 frases e algumas perguntas; um modelo pequeno basta, custa menos e varia pouco entre uma chamada e outra.
- **Limites:** 15 s por tentativa e 2 tentativas. Se a API travar, o erro chega em uns 30 s e a tela não fica girando.
- **Sem memória, de propósito:** cada raio-x é independente. Memória de conversa só traria o risco de misturar dados de uma empresa na outra.
- **Sem agente, de propósito:** o caminho é sempre o mesmo (buscar → calcular → escrever → validar). Não há decisão para um agente tomar, e um pipeline fixo é mais previsível e mais fácil de testar.
- **Plano B sem IA:** `texto_sem_ia()` monta o texto por modelo fixo, com os mesmos marcadores e o mesmo validador. A demo funciona sem a chave do Gemini.

---

7. Ética, governança e segurança

[PENDENTE: texto final · NA5, Náthally, sáb 07 a dom 08/11] *Esta seção é a base técnica (`docs/etica.md`), atualizada em 04/11. A Náthally reescreve para a banca e completa a viabilidade.*

7.1 O risco principal: número inventado

O LLM pode inventar ou arredondar um número, e o vendedor levaria um dado falso para a reunião. Por isso o LLM só escreve marcadores e o validador (`checar()` e `validar_e_preencher()`, `raiox/ia.py`) barra algarismos de qualquer escrita (inclusive ½ e ²), números por extenso ("dez", "o dobro") e marcadores inventados. Se o resumo tiver um número, ele sai inteiro; perguntas e lacunas saem uma a uma; tudo o que sai fica registrado. **Resultado:** nos 10 raio-x da avaliação, 0 trechos removidos (seção 8).

7.2 O que o validador não pega

O validador barra números, não ideias. Continuam possíveis:

- **pergunta com suposição falsa** (ex.: supor que os custos explicam uma margem baixa);
- **"acima" ou "abaixo" trocado na frase:** a posição é calculada por código, mas a palavra na frase é do LLM. O que protege: a regra do prompt e o cartão de cada indicador na tela, que mostra a posição calculada;
- **lacuna genérica** ("quem decide a compra" serve para qualquer empresa);
- **interpretação de setor:** o modelo pode associar a empresa a um negócio que ela não tem.

O que resolve, no fim, é a leitura humana. Por isso o raio-x é apoio, e não decisão (7.6).

7.3 Viés e limites do modelo de ML

- **Só companhias abertas:** empresas fechadas, pequenas ou médias não aparecem. A ferramenta serve para grandes contas.
- **Empresas pequenas:** as 31 com receita abaixo de R$ 10 mi recebem um grupo sem terem ajudado a formá-lo; para elas, a comparação é menos confiável.
- **Extremos cortados só no modelo:** os números mostrados são os reais, mas uma empresa muito fora da curva fica num grupo que não a representa bem.

- **Perfil financeiro, não setor:** as parecidas da WEG (máquinas) incluem Cemig e Copel (energia), Votorantim Cimentos e Telefônica: empresas grandes, rentáveis e pouco endividadas, mas de outros setores. Isso serve para prospectar perfis parecidos, mas não são concorrentes.
- **Silhueta 0,29:** os grupos se sobrepõem. É uma régua de comparação, não uma classificação exata.

7.4 Confiabilidade dos dados

- **Dado de até um ano atrás:** a DFP é anual; resultado trimestral não entra. O ano aparece em cada número.
- **Fonte em cada número:** cada indicador mostra a conta da DFP de onde veio, para o vendedor conferir.
- **Conferência contra o RI** (Marcos, 03/11, `avaliacao/conferencia.csv`): 13 valores de 5 empresas (WEG, Camil, Magalu, RD Saúde e CSN), todos com diferença de 0,00%.
- **A conferência manual de 29/10** (Náthally) deu 7 de 10. Das 3 diferenças: a Camil era um erro nosso (o ano fiscal), corrigido no PR #46; em Magalu e RD Saúde, o sistema bate com o RI, e a diferença parece vir da fonte usada na conferência manual (não se sabe qual fonte foi usada nela).

7.5 Segurança de dados

- **Chave do Gemini** só no `.env`, fora do Git. No Terraform, o `.env` é montado só para leitura e a chave não passa por variável, então não fica no `terraform.tfstate`. Nenhuma chave em todo o histórico do Git (busca pelo padrão de chave, 29/10).
- **Ponto fraco conhecido:** a senha do PostgreSQL local passa por variável do Terraform e fica no `terraform.tfstate`, que não vai para o Git. O banco só aceita conexões do próprio computador (`127.0.0.1`).
- **A tela no `docker compose`** abria para a rede toda; o Terraform já fechava em `127.0.0.1`. Correção no PR #47 (issue #44). **[PENDENTE: confirmar o merge do #47]**
- **O que vai para o Gemini:** o setor, os indicadores já formatados com a posição, o tamanho do grupo, o produto e o objetivo. Nem o nome da empresa vai (`montar()`, `raiox/ia.py`). **[PENDENTE: citar a política de uso de dados do Google para a API paga]**
- **O que fica gravado:** a tabela `consultas` guarda empresa, produto, objetivo, tempo e custo. Não há dado pessoal, mas o produto e o objetivo são texto livre: o vendedor não deve escrever nome de pessoa ou dado sigiloso ali.
- **Travas na IA de programação:** a IA que ajudou a escrever o código não lê o `.env`, não faz commit de dados nem de chaves e pergunta antes do merge (seção 9).

7.6 Uso responsável

- **Apoio, não decisão:** o raio-x prepara a primeira reunião; o vendedor confere antes de usar. A tela avisa: "Confira antes de usar".
- **Não é recomendação de investimento:** o mesmo aviso, na tela.
- **Se a IA falhar, a ferramenta continua:** a tela monta o texto sem IA, com os mesmos números, e avisa. Conferido em 03/11 com uma chave inválida: WEG gerada sem IA e consulta gravada com custo 0.
- **Custo acompanhado:** cada consulta grava os tokens e o custo.

7.7 Viabilidade

- **Custo de IA por raio-x:** ≈ US$ 0,0012 (seção 8).
- **Tempo de resposta da IA:** 2,2 s em média.
- **Manutenção:** a DFP é anual; a base é recarregada uma vez por ano com 2 comandos (`scripts/carregar_dfp.py` e `scripts/indicadores.py`).
- **[PENDENTE: ROI · NA3, Náthally]**

8. Métricas

8.1 Avaliação com 10 casos reais (`avaliacao/resultados.csv`)

10 raio-x com o Gemini, um por caso de `avaliacao/casos_avaliacao.csv` (empresa, produto e objetivo escritos pela Náthally): Camil, CSN, Magazine Luiza, Petrobras, Embraer, Suzano, Raia Drogasil, Localiza, WEG e Telefônica. Rodados em 29/10 (PR #20).

métrica	resultado
raio-x gerados	10 de 10
trechos removidos pelo validador	0
custo total	US$ 0,0118
custo por raio-x	US$ 0,0012 em média (de 0,0011 a 0,0012)
tempo de resposta da IA	2,2 s em média (de 1,8 a 2,9 s)
tokens por raio-x	≈ 2.262 de entrada e ≈ 409 de saída, em média

O custo é calculado com os tokens reais de cada chamada e o preço da página oficial do Google (US$ 0,25 por milhão de tokens de entrada e US$ 1,50 por milhão de saída, conferido em 20/10/2026).

Os 10 raio-x foram gerados antes da correção do ano fiscal (PR #46): o da Camil usou o exercício anterior. O custo, o tempo e os trechos removidos não dependem disso. **[PENDENTE: se a avaliação for refeita na `v1.0` (ED11), trocar os números]**

8.2 Conferência dos números contra o RI (`avaliacao/conferencia.csv`)

empresa	valores conferidos	diferença
WEG	receita, EBIT, lucro líquido, ativo total, patrimônio líquido	0,00%
Camil	receita (exercício de mar/2025 a fev/2026)	0,00%
Magazine Luiza	receita, EBIT	0,00%
RD Saúde	receita, EBIT, ativo total, patrimônio líquido	0,00%
CSN	receita	0,00%

13 valores de 5 empresas, cada um com o documento do RI e a página de onde veio (coluna `fonte` do arquivo).

8.3 Qualidade do código

- Testes automáticos: 241 passando, nenhum pulado (03/11, com o PostgreSQL de testes). Os testes não chamam a API.
- O CI roda os testes em todo pull request, com um PostgreSQL só para eles; a `main` só recebe mudanças por PR com o CI verde.

9. Como usamos IA para programar (vibecoding)

O time é iniciante: três pessoas programaram com o Claude Code (o Miguel começou com o Codex) e a Náthally não usou IA para código. A IA escreveu muito do código; **quem responde por ele é a pessoa**. Para isso valer, todo trabalho seguiu o mesmo caminho:

1. **Começo:** o comando `/inicio-de-sessao` cria uma branch a partir da `main` e faz a IA ler os arquivos do professor e o estado do projeto antes de mexer em qualquer coisa.

2. **Durante:** a IA segue o `CLAUDE.md` (donos de cada pasta, regra de ouro, contratos) e o `REGRAS-IA.md` (o que nunca vai para uma IA: chave, senha, dados pessoais). Toda afirmação sobre o código vem com arquivo e linha, ou com um comando que prova.
3. **Fim:** `/fim-de-sessao` roda os testes, atualiza as notas do projeto e abre o PR. O CI roda os testes de novo; um segundo Claude (o `revisor`), que não escreveu o código, lê o diff; o merge só acontece depois de um "pode" da pessoa.
4. **Travas:** as regras que custam caro quebrar viraram bloqueios automáticos (`.claude/settings.json` e `.claude/hooks/guarda.py`): a IA não lê o `.env`, não apaga trabalho no Git, não faz commit na `main` nem com dados ou chaves, e pergunta antes do merge. Na estreia, o `revisor` achou um jeito de vazar a chave que o autor não tinha visto; virou teste.
5. **No PR, duas linhas obrigatórias:** o que foi feito com IA e o que o autor entendeu ou conferiu.

Os arquivos de prompts do vibecoding (`requisitos_projeto.MD`, Aplicação) estão no repositório e listados no `README.md`: `CLAUDE.md`, `AGENTS.md`, `REGRAS-IA.md`, `.claude/skills/`, `.claude/agents/revisor.md`, `.claude/settings.json`, `.claude/hooks/guarda.py`, `.githooks/pre-commit` e `.github/pull_request_template.md`. O prompt do produto (o que o Gemini recebe) é outro: `avaliacao/regras_prompt.md`.

---

10. Lições aprendidas

- **Número bonito pode enganar:** a silhueta de 0,77 media só a separação de 9 empresas extremas.
- **Teste que passa nem sempre testa:** dois testes passavam com o campo vazio. Passamos a estragar o código de propósito para ver o teste falhar.
- **Teste pulado não é teste que passou:** o CI ficou verde com 17 testes de banco pulados, porque não havia PostgreSQL nele. Agora o CI sobe o banco e lista os pulados.
- **A fonte vale mais que o resumo da fonte:** uma nota interna resumia o guia e virou "verdade", até para a IA, que disse que o Terraform não era preciso. Os arquivos do professor passaram a ficar no repositório e a ser lidos em toda sessão.
- **Branch longa vira retrabalho:** três vezes alguém teve de trazer à mão o que estava numa branch antiga. Agora toda sessão começa da `main`.
- **Contrato antes de código:** os erros mais caros (CSV com `;`, código CVM como texto) foram de formato entre as partes.

Apêndice A · De onde vem cada número

número neste documento	fonte	como conferir
595 empresas com indicadores	`data/indicadores.csv` (gerado, fora do Git)	`python -c "import pandas as pd; print(len(pd.read_csv('data/indicadores.csv')))"`
384.591 linhas de contas; carga em 1 min 48 s	carga do Marcos em 03/11 (`docs/REQUISITOS.md`, R2)	`python scripts/carregar_dfp.py` imprime o total
k = 4; silhueta 0,29; grupos 327, 76, 59 e 102; 564 no treino	`segmentar()` com os dados de 30/10	`python -m raiox.segmentos`
567 de 595 num grupo só; silhueta 0,77 antes	ED1 e ED4 (27 e 28/10), `docs/HISTORICO.md`	`docs/graficos/silhueta_antes_ed4.png`
31 empresas abaixo de R$ 10 mi (52%); 564 acima (4%)	`scripts/avaliar_corte_receita.py`, 03/11	rodar o script

número neste documento	fonte	como conferir
receita da Camil: R$ 12,26 bi → R$ 11,12 bi; 9 empresas mudaram	PR #46	`avaliacao/conferencia.csv`, linha da Camil
nenhum "erro" e 180 "atenção" em 98 empresas	`raiox/checagem.py`, 03/11	`python -m raiox.checagem data/indicadores.csv data/contas.csv`
10 raio-x, 0 removidos, US$ 0,0118, US$ 0,0012, 2,2 s, tokens	`avaliacao/resultados.csv`	soma e média das colunas `n_removidos`, `custo_usd`, `tempo_s`, `tokens_*`
US$ 0,25 e US$ 1,50 por milhão de tokens	`PRECO_US_POR_MILHAO`, `raiox/ia.py`	página oficial de preços do Google (20/10/2026)
13 valores de 5 empresas, 0,00%	`avaliacao/conferencia.csv`	coluna `diferenca_pct`
7 de 10 na conferência manual	Náthally, 29/10 (`docs/HISTORICO.md`)	`revisao_manual.md`, na branch `feat/metricas`
WEG com o Gemini: US$ 0,0011	demo de 03/11 (`docs/HISTORICO.md`)	tabela `consultas` do banco da demo
15 s por tentativa, 2 tentativas, `temperature=0.2`	`TEMPO_LIMITE_S`, `TENTATIVAS`, `raiox/ia.py`	ler o arquivo
banco desiste em 5 s	`TEMPO_LIMITE_S`, `raiox/banco.py`	ler o arquivo
241 testes passando, nenhum pulado	Windows do Marcos, 03/11, com o PostgreSQL (`docs/PLANO.md`, seção 0)	`python -m pytest -q -rs` com o PostgreSQL ligado

Apêndice B · Pendências deste rascunho (sai da versão final)

o que falta	de quem	quando
BPMN 2.0 (`docs/bpmn/*.png`) na seção 4	Marcos (MA5)	qui 05/11
diagrama de arquitetura (`docs/arquitetura.png`) na seção 5	Miguel (MG5)	qua 04/11
texto final de ética na seção 7 e a viabilidade	Náthally (NA5)	sáb 07 a dom 08/11
ROI na seção 1 e em 7.7	Náthally (NA3)	qui 05/11
gráficos da EDA na seção 3	Marcos (MA4)	—
gráficos do K-means refeitos com os dados depois do PR #46	Eduardo	sex 06/11
números do Apêndice A conferidos de novo na `v1.0` (recarga da CVM depois do #46)	Eduardo	seg 09/11
merge do PR #47 (tela fechada em `127.0.0.1` no compose)	Miguel	—
política de uso de dados do Google para a API paga (7.5)	Náthally ou Eduardo	seg 09/11
datas reais do congelamento e da `v1.0` (seção 2)	Eduardo	seg 09/11